Manual do Projeto Prático: Sistemas Distribuídos

Engenharia de Software, Concorrência e Tolerância a Falhas

Prof. Dr. Leonardo Grando

1º Semestre de 2026

1. Visão Geral do Projeto

O Conceito: "UniDriver" (Modelo de Referência)

Neste semestre, não faremos apenas provas teóricas. Faremos Engenharia de Sistemas. Para guiar nosso aprendizado, usaremos como base o **UniDriver** (um clone simplificado de transporte por aplicativo).

- **O Problema:** Conectar passageiros (Clientes) a um recurso escasso (Motorista) através de uma rede não confiável.
- **O Desafio:** O sistema deve funcionar em rede, evitar conflitos (dois passageiros pegando o mesmo carro) e sobreviver a falhas do servidor.

Liberdade Criativa (Escopo Aberto)

O "UniDriver" é o exemplo didático. Seu grupo **PODE** e **DEVE** adaptar este modelo para um tema de seu interesse, desde que mantenha os desafios técnicos.

Sugestões de Temas:

1. **Jogos Multiplayer:** (Ex: Batalha Naval). Recurso Crítico: O Tabuleiro.
2. **Sistema de Leilão:** Recurso Crítico: O Valor do Lance Atual.
3. **Chat Corporativo:** Recurso Crítico: O Histórico de Mensagens.
4. **IoT:** Controle de Sensores em tempo real.

Fiquem à vontade para escolher o tema.

---

2. Cronograma de Entregas (Sprints)

O projeto é dividido em 4 entregas obrigatórias, alinhadas com a teoria da aula.

Sprint	Tema Técnico	O que deve ser entregue?	Data Limite
0	**Definição**	Definição do Tema e do Recurso Crítico.	03/03
1	**Comunicação**	Cliente e Servidor trocando JSON via Sockets em máquinas diferentes.	17/03
2	**Concorrência**	Servidor Multithread com **Mutex (Lock)** para evitar conflitos de acesso.	31/03

Sprint	Tema Técnico	O que deve ser entregue?	Data Limite
3	**Ordenação**	Implementação de **Relógios Lógicos (Lamport)** e Logs de Auditoria.	28/04
4	**Resiliência**	**Tolerância a Falhas**. O Cliente deve reconectar automaticamente se o servidor cair.	02/06

---

3. Guia de Implementação (Tutorial)

Para facilitar o início, utilizaremos **Python + Sockets + JSON**.

- **Sockets:** O "telefone" que conecta as máquinas.
- **JSON:** O formato da mensagem (Marshalling).

Passo 1: O Servidor (`servidor.py`)

Este código implementa um servidor que aceita conexões, usa Threads para atender vários clientes e possui um Lock para proteger o motorista.

```
import socket
import threading
import json
import time

# --- CONFIGURAÇÕES ---
HOST = '0.0.0.0'  # Ouve em todas as interfaces de rede
PORT = 5000       # Porta de comunicação

# --- ESTADO GLOBAL ---
motorista_disponivel = True
# --- MUTEX (SPRINT 2) ---
lock_motorista = threading.Lock()

def atender_cliente(conn, endereco):
    global motorista_disponivel
    print(f"[{endereco}] Conectado.")

    try:
        while True:
            dados = conn.recv(1024)
            if not dados: break

            # Unmarshalling (Bytes -> Dicionário)
            mensagem = json.loads(dados.decode('utf-8'))
            acao = mensagem.get('acao')

            resposta = {}

            # --- ZONA CRÍTICA (SPRINT 2) ---
            if acao == 'SOLICITAR_CORRIDA':
                # O "with" adquire o Lock e solta no final automaticamente
```

```
                with lock_motorista:
                    if motorista_disponivel:
                        print(f"Processando pedido de {endereco}...")
                        time.sleep(2) # Simula processamento
                        motorista_disponivel = False
                        resposta = {"status": "ACEITO", "msg": "Motorista a caminho!"}
                        # Libera motorista após 10s
                        threading.Timer(10, liberar_motorista).start()
                    else:
                        resposta = {"status": "NEGADO", "msg": "Motorista ocupado."}
            else:
                resposta = {"status": "ERRO", "msg": "Comando desconhecido"}

            # Marshalling e Envio
            conn.sendall(json.dumps(resposta).encode('utf-8'))

    except Exception as e:
        print(f"Erro: {e}")
    finally:
        conn.close()

def liberar_motorista():
    global motorista_disponivel
    with lock_motorista:
        motorista_disponivel = True
        print("Motorista liberado!")

def iniciar():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # CORREÇÃO DE ERRO "ADDRESS ALREADY IN USE"
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen()
    print(f" Central UniDriver OUVINDO na porta {PORT}...")

    while True:
        conn, addr = s.accept()
        t = threading.Thread(target=atender_cliente, args=(conn, addr))
        t.start()

if __name__ == "__main__":
    iniciar()
```

Passo 2: O Cliente (`cliente.py`)

Este código implementa a **Tolerância a Falhas** (Sprint 4). Ele tenta reconectar infinitamente.

```
import socket
import json
import time

HOST = 'localhost' # Coloque o IP do servidor aqui se for em outra máquina
PORT = 5000

def conectar():
    while True: # Loop de Reconexão (Sprint 4)
        try:
```

```
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.connect((HOST, PORT))
            print(" Conectado! Pressione ENTER para pedir corrida.")

            while True:
                input()
                pedido = {"acao": "SOLICITAR_CORRIDA", "passageiro": "Aluno"}
                s.sendall(json.dumps(pedido).encode('utf-8'))

                dados = s.recv(1024)
                if not dados: break

                resp = json.loads(dados.decode('utf-8'))
                print(f"Resposta: {resp['status']} - {resp['msg']}")

        except (socket.error, ConnectionRefusedError):
            print(" Servidor indisponível. Reconectando em 3s...")
            time.sleep(3)
        finally:
            s.close()

if __name__ == "__main__":
    conectar()
```

---

4. Ferramenta de Teste: O Script de Ataque

Para testar se o seu **Mutex (Lock)** está funcionando na Sprint 2, use este script. Ele simula 5 passageiros clicando no botão ao mesmo tempo.

Arquivo: `ataque.py`

```
import socket
import json
import threading

HOST = 'localhost'
PORT = 5000

def atacar(id):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((HOST, PORT))
        msg = {"acao": "SOLICITAR_CORRIDA", "passageiro": f"Bot_{id}"}
        s.sendall(json.dumps(msg).encode('utf-8'))
        resp = json.loads(s.recv(1024).decode('utf-8'))
        print(f"[Bot {id}] Resultado: {resp['status']}")
        s.close()
    except: pass

print("INICIANDO TESTE DE CARGA...")
threads = []
for i in range(5):
    t = threading.Thread(target=atacar, args=(i,))
    threads.append(t)
    t.start()
```

Como Interpretar:

- **Correto:** Apenas 1 Bot recebe "ACEITO". Os outros 4 recebem "NEGADO".
- **Errado (Race Condition):** Mais de um Bot recebe "ACEITO". (Significa que vocês esqueceram o Lock!).

---

5. Critérios de Avaliação (Checklist)

O projeto compõe **60%** da nota bimestral (ADC2). A Participação Individual compõe **40%**.

Parte A: Código do Grupo (0 a 10)

Critério	Peso	O que será avaliado?
Arquitetura Distribuída	2.0	Cliente e Servidor são processos separados? Usam Sockets/JSON corretamente?
Concorrência (Mutex)	3.0	O sistema passou no "Script de Ataque"? O dado se manteve íntegro?
Ordenação (Logs)	2.0	Existe um Log de Eventos? Ele respeita a ordem causal (não apenas temporal)?
Tolerância a Falhas	3.0	Se o professor desligar o servidor, o cliente reconecta sozinho?

---

6. Diretrizes para o Relatório Técnico

A entrega final deve ir além do código. Vocês devem entregar um **Relatório Técnico** que comprove o entendimento da engenharia aplicada.

Formato de Entrega: O relatório deve ser o arquivo `README.md` principal do repositório no GitHub (estilo Portfólio) OU uma página Web documentada (GitHub Pages). **Não serão aceitos arquivos .DOCX.**

Estrutura Obrigatória do Relatório:

1. **Identificação:** Nome do Projeto e Integrantes.
2. **Arquitetura do Sistema:**

- Um diagrama simples mostrando quem é o Cliente, quem é o Servidor e como eles se conectam (pode ser feito no Draw.io, Mermaid ou Excalidraw).

3. **Protocolo de Comunicação:**

- Exemplo do JSON utilizado (quais campos vocês criaram?).

4. **Desafios de Concorrência:**

- Explique qual é o "Recurso Crítico" do seu projeto.
- Mostre um print/snippet do código onde o `Lock/Mutex` foi implementado.

5. **Evidências de Teste (Screenshots):**

- Print do "Script de Ataque" rodando e provando que o sistema bloqueou os acessos simultâneos.
- Print do "Log de Auditoria" mostrando os Relógios de Lamport.
- Print do Cliente reconectando após a falha do servidor.

6. **Como Rodar:**

- Passo a passo simples para o professor clonar e rodar o projeto.

Parte B: Defesa Individual (0 a 10)

- O professor fará perguntas direcionadas a cada membro do grupo sobre o código.
- **Exemplo:** "Fulano, mostre a linha onde o Lock é liberado e explique o que acontece se der erro antes disso."
- *Quem não souber explicar o código do próprio grupo terá a nota reduzida.*